

MidTool: Mid-training Data Synthesis for Agentic Tool Use

Fengqing Jiang^{1,†}, Yite Wang², Boyi Liu², Zhaoyang Wang^{3,†}, Canwen Xu², Zhewei Yao², Radha Poovendran^{1,‡}, Yuxiong He^{2,‡}

¹University of Washington ²Snowflake ³University of North Carolina at Chapel Hill

[†]Work done at Snowflake [‡]Co-advising

Mid-training is increasingly recognized as a critical stage for shaping the capabilities of large language models. Recent work has shown that targeted mid-training can strengthen reasoning-intensive abilities such as math and science, and can also improve agentic capabilities in software-engineering settings. In this work, we study the parallel but less explored agentic capability: general tool use. We present MidTool, an open corpus construction pipeline for agentic tool-use mid-training that combines large-scale web, PDF, and code data with synthesized supervision from real-world tool APIs, MCP skills, and document-grounded workflows. MidTool is designed to teach models how to recognize tool affordances, ground arguments from context, compose tool call workflow, and recover from incomplete information. We mid-train Qwen3-4B-Base and Qwen3-8B-Base on MidTool-Mix, and then apply follow-up post-training with both supervised fine-tuning and reinforcement learning. Compared with baselines, MidTool-Mix consistently improves downstream performance under both SFT and RL on BFCL, τ^2 -Bench, and MCP Universe. These results suggest that general tool use, like other important LLM capabilities, benefits from dedicated mid-training rather than being left entirely to post-training.

Contact: yite.wang@snowflake.com

Data & Model: <https://hf.co/collections/MidTool/midtool-release>  snowflake

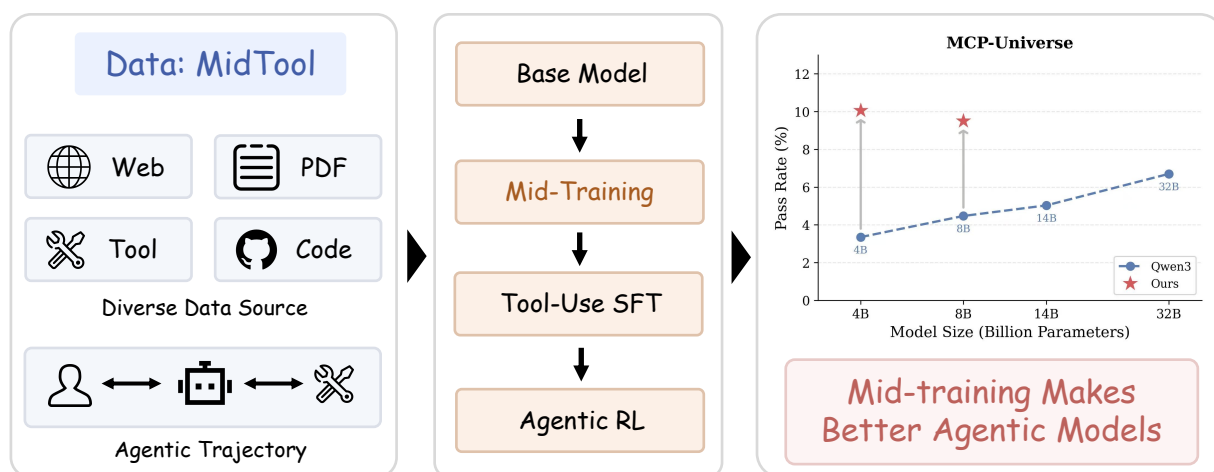


Figure 1 | **Left:** MidTool-Mix is a 20.3B-token mid-training corpus built from web, PDF, code, and tool sources with synthesized agentic trajectories. **Middle:** Training pipeline from base model. **Right:** On MCP-Universe, our mid-trained 4B and 8B models outperform Qwen3 official models, showing that dedicated mid-training yields stronger agentic capability than scaling alone.

1. Introduction

Tool use is becoming a defining capability of large language model (LLM) agents. Strong systems must decide when external tools are needed, ground calls in tool schema, extract arguments from long and noisy context, compose multiple tools into workflows, and recover when information is incomplete. These behaviors now matter across function calling, API orchestration, interactive agent benchmarks, and emerging Model Context Protocol (MCP) ecosystems (Liu et al., 2024; Luo et al., 2025; Patil et al., 2025; Schick et al., 2023).

Most recent progress on tool use, however, has come from post-training. Supervised fine-tuning (SFT) and reinforcement learning (RL) on curated traces have substantially improved function calling and agent behavior (Liu et al., 2025; Prabhakar et al., 2025; Song et al., 2024; Xu et al., 2025), but they also place a heavy burden on post-training. The model must simultaneously acquire a set of atomic agentic capabilities, including tool recognition, schema-grounded argument construction, clarification under missing information, and multi-step execution, from comparatively narrow supervision. More fundamentally, the knowledge that underlies successful tool use does not live only in explicit trajectories. It is distributed across developer documentation, manuals, PDFs, code repositories, API specifications, and structured tool definitions, most of which never appear as clean agent demonstrations (Liu et al., 2024; Qin et al., 2023; Xu et al., 2025).

This raises a natural question: **can general tool-use ability be shaped earlier through dedicated mid-training, rather than being left almost entirely to post-training?** Mid-training is a distinct stage of multi-stage pre-training, bridging the gap between general pre-training and post-training stages (Tu et al., 2025). Recent work has shown that targeted mid-training can materially improve reasoning-intensive capabilities (Team et al., 2025a; Wang et al., 2025), and emerging efforts have begun to explore agentic mid-training for deep research (Su et al., 2025; Team et al., 2025b), coding (Xiao et al., 2026; Z.ai, 2026; Zeng et al., 2025), and software engineering (SWE) (Yang et al., 2025; Zeng et al., 2026). Yet general tool use remains underexplored as a mid-training target. Compared with math or SWE tasks, tool use data requires covering a broader and more heterogeneous capability surface, including natural-language documentation, executable code patterns, structured schemas, multi-tool workflows, and failure cases caused by missing information.

In this paper, we introduce MidTool, a scalable pipeline for constructing agentic mid-training data of general tool use, together with MidTool-Mix, the resulting 20.3B-token mixture. Our pipeline begins from four complementary source families: web pages, PDFs, code repositories, and structured tool artifacts such as APIs and MCP skills. It then converts these sources into training supervision through two synthesis branches that target the two core deficits of tool use: grounding and execution. The first branch, **context-grounded trajectory augmentation**, addresses grounding by turning documentation and code into supervision for recognizing tool boundaries, inferring parameters, and recovering workflow structure from messy real-world artifacts. The second branch, **native agentic trajectory synthesis**, addresses execution by constructing executable trajectories directly from real APIs, MCP skills, and collected rollout traces, teaching multi-turn planning, clarification, and recovery with explicit validation of schema grounding, turn order, required arguments, and tool-response consistency. The resulting mixture is designed to teach models not only what tools exist, but also the atomic agentic capabilities needed to recognize their affordances, compose them into workflows, and recover when information is incomplete. Table 1 situates our work among representative open corpora and mid-training efforts. Relative to prior open efforts, our goal is neither a general-domain mixture nor a domain-specific corpus for math, deep research, or software engineering, but a dedicated agentic corpus for *general tool use*.

We evaluate whether such mid-training provides value beyond standard post-training by mid-training

Work	Domain	Data Size	Train	Data Sources	Tool Div.	Agent Traj.	Public Access
FineWeb (Penedo et al., 2024)	General	15T	PT	Web	○	✗	✓
Dolmino (Olmo et al., 2025)	General	100B	MT	Crawl sourced + Synthesized	○	✗	✓
MegaMath-Web-Pro (Wang et al., 2025)	Math	100B	MT	Math, QA, Instruction + Synthesized	○	✗	✓
AgentFounder (Su et al., 2025)	Deep Research	300B	MT	Web + Synthesized	◐	✓	✗
daVinci-Dev (Zeng et al., 2026)	SWE	73.1B	MT	Github PR + Synthesized	◐	✓	✓
MidTool-Mix (ours)	Tool Use	20.3B	MT	Web, PDF, Code, Tool + Synthesized	●	✓	✓

Note: ○ low/none, ◐ partial, and ● high; ✓ and ✗ denote yes and no, respectively.

Table 1 | Comparison of representative work on pre-training (PT) and mid-training (MT) corpora. Tool Div. = tool diversity and Agent Traj. = (include) agentic trajectories.

Qwen3-4B-Base and Qwen3-8B-Base on MidTool-Mix, followed by the same downstream SFT and optional RL recipes. Across BFCL (Patil et al., 2025), τ^2 -Bench (Barres et al., 2025), and MCP-Universe (Luo et al., 2025), we observe a consistent pattern: MidTool-Mix improves downstream tool-use performance over SFT-only baselines, and RL usually compounds these gains. The gains are especially pronounced on harder multi-turn and interactive settings.

Contributions. Our main contributions are threefold: **(1)** To our knowledge, we introduce MidTool, the first open pipeline and mid-training dataset designed for general tool use, and construct MidTool-Mix, a 20.3B-token mixture that combines web, PDF, code, and tool artifacts with both context-grounded augmentation and native agentic trajectories. **(2)** We show that dedicated tool-use mid-training consistently improves downstream general tool-use performance for 4B and 8B models across three benchmarks under both SFT and RL, suggesting that mid-training provides a stronger and more stable substrate for subsequent post-training. **(3)** We show that MidTool-Mix helps reveal a meaningful capability boundary: its benefits are strongest for broad tool-use capability and transfer, while more specialized exploratory behaviors remain distinct. This suggests that capabilities such as deep search likely require dedicated mid-training data beyond a purely general tool-use mixture, providing concrete guidance for future agentic mid-training study.

2. MidTool: Scalable Pipeline for Agentic Mid-training Data Synthesizing

In this section, we present our pipeline to construct the large-scale mid-training corpus, including data source collection, data preprocessing and agentic trajectory synthesis stages.

2.1. Stage 1: Data Source Collection

As illustrated in Figure 2, our data collection stage is designed to cover the complementary signals required for agentic tool use. Effective tool-use behavior depends not only on explicit trajectories, but also on broad technical knowledge, tool-facing documentation, executable code patterns, and grounded tool schemas. We therefore collect four source families: web documents, PDFs, code repositories, and structured tool artifacts.

Web data. We use processed Common Crawl dumps from FineWeb (Penedo et al., 2024) as the primary large-scale web source. We sample multiple dumps spanning 2020 to 2025 to capture recent technical content, including API references, developer documentation, troubleshooting pages, tutorials, and CLI-style instructions. Relative to later trajectory synthesis, the web corpus provides broad coverage of tool-related concepts, terminology, and workflow descriptions at scale.

PDF data. Many high-value tool-use resources, such as manuals, product handbooks, and platform

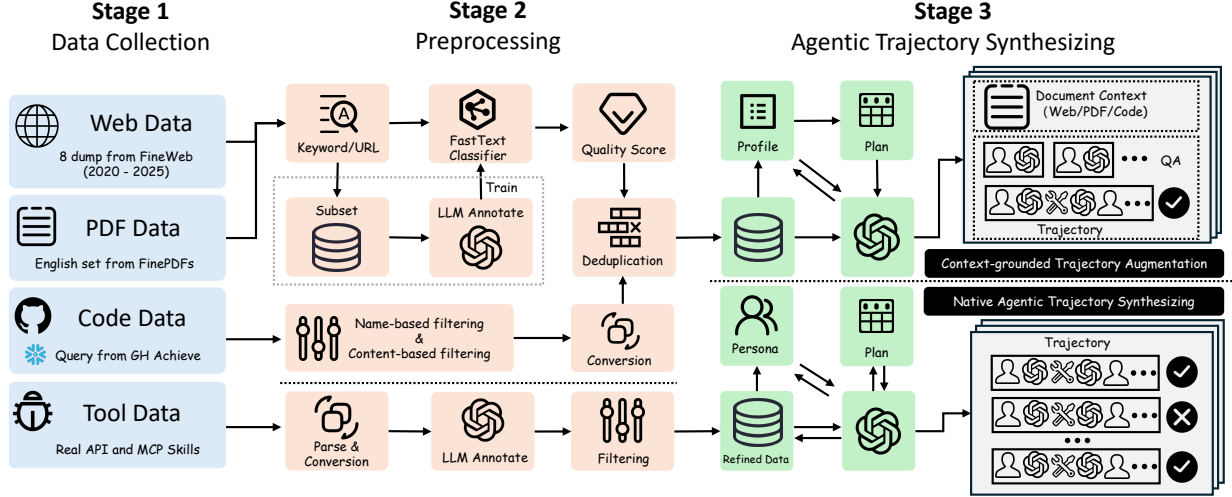


Figure 2 | Overview of MidTool pipeline. Stage 1 collects four complementary source families, including web pages, PDFs, code repositories, and structured tool artifacts. Stage 2 applies source-specific preprocessing, quality control, and deduplication. Stage 3 converts the refined corpus into executable supervision through two branches: context-grounded trajectory augmentation from web/PDF/code documents, and native agentic trajectory synthesis from real APIs and MCP skills.

documentation, are distributed as PDFs rather than clean HTML pages. We therefore use FinePDFs (Kydlíček et al., 2025) as a complementary crawl source and keep only the English subset. Compared with web pages, PDFs contribute longer-form procedural content and documentation that is often absent from standard web snapshots, but they also require stricter downstream filtering because extraction noise is substantially higher.

Code data. Tool use is tightly coupled with software artifacts, so we collect repository data from two complementary GitHub slices. We initialize repository discovery from Snowflake GH Archive event data (GH Archive, 2026), then apply task-specific filtering. The first slice targets agent-related and MCP-related repositories obtained from GitHub event data, with filtering for recency, activity, and selective licensing. This slice captures concrete tool interfaces, orchestration patterns, and emerging agent engineering practice. The second slice targets high-quality public repositories with strong community signals and recent activity in major programming ecosystems such as Python, Java, JavaScript, TypeScript, Go, Rust, C/C++, C#, SQL, Shell, and Dockerfile. We further retain repositories that are likely to contain useful libraries, SDKs, frameworks, examples, or educational technical content, while removing personal projects, forks, and benchmark or dataset repositories that would add noise or leakage risk. In particular, we explicitly exclude known benchmark and evaluation repositories from the GitHub code slice through an actively maintained blacklist during data collection; more details are provided in Appendix A.1.

Tool data. Finally, we separately collect structured tool artifacts, including REST APIs and MCP skills, to support native agentic trajectory synthesis. Unlike web, PDF, and code sources, these artifacts expose executable schemas, parameter structures, and tool boundaries directly. They therefore provide the most faithful substrate for constructing grounded tool calls, multi-tool plans, and recovery behaviors in the later synthesis stage.

Taken together, these four source families intentionally trade off breadth and structure. Web and PDF data provide broad technical context, code repositories contribute executable patterns and developer workflows, and tool artifacts supply explicit schemas and callable interfaces. The subsequent preprocessing and synthesis stages convert this heterogeneous raw corpus into a unified training mixture for agentic mid-training.

2.2. Stage 2: Data Preprocessing

Code. Following [Li et al. \(2023\)](#) and [Weber et al. \(2024\)](#), we apply multi-phase filtering to construct a high-value subset for effective training. We adapt the extension-based filtering list to exclude files such as binary data, model weights, and logs. We then adapt the heuristics from StarCoder ([Li et al., 2023](#)) for content-based filtering, including line count, average line length, maximum line length, and alpha ratio. We also convert Jupyter Notebook files to Python text files. For deduplication, we remove exact duplicates by SHA-256 hashing over normalized text, and we remove near-duplicates with MinHash LSH. For accepted high-quality repositories, we apply a filter that matches only documentation-like directories such as docs, examples, tutorials, guides, samples, and cookbook.

Web and PDF data. For web and PDF midtraining sources, we use a four-phase pipeline: high-recall keyword/URL prescreening, a lightweight fastText ([Joulin et al., 2017](#)) classifier trained on LLM-labeled seed data, document-level quality filtering, and MinHash LSH deduplication. The pipeline is designed to retain developer-oriented documentation and technical reference material while suppressing generic web noise and PDF extraction artifacts, with stricter thresholds for PDFs. Full details are provided in Appendix A.2.

2.3. Stage 3: Agentic Trajectory Synthesizing

As shown in our later study, effective tool-use mid-training must address two complementary deficits. The first is *grounding*: models often fail to infer tool boundaries, required arguments, and workflow structure from messy real-world artifacts such as documentation, PDFs, and code. The second is *execution*: even when tool schemas are available, models still struggle to plan across multiple turns, request missing information, sequence calls correctly, and recover from incomplete interactions. We therefore instantiate a two-branch synthesis pipeline that converts both unstructured documents and structured tool artifacts into normalized agent trajectories.

Context-grounded Trajectory Augmentation. Many web, PDF, and code documents expose tool affordances and workflows without containing explicit interaction traces, so we use them as the grounding-oriented branch. We first apply a lightweight keyword-based prefilter, then use Qwen3-235B-A22B-Instruct-2507 to annotate each remaining document with quality score, and a structured affordance profile. The affordance profile records whether tool responses can be inferred, together with evidence about schema/API structure, code or CLI usage, workflow structure, tool topology, and domain terminology. Documents below a quality threshold are kept without augmentation, while for the retained documents, a rule-based planner converts the extracted affordances into a synthesizing plan.

The planner ties supervision volume to document quality, allocating a bounded budget for diverse QA types and allowing at most one multi-turn chain per document. The regular QA decomposes tool use into atomic agentic capabilities, such as tool selection, schema-grounded parameter extraction, format-constrained calls, workflow recognition, and multiple/parallel use, while the trajectory samples cover sequential execution, parameter clarification, tool switching, and long-context reasoning. We use Qwen3-235B-A22B-Instruct-2507 to synthesize the augmented data. Only QA pairs and trajectories that pass parsing and semantic quality control are merged into the final training mix.

Native Agentic Trajectory Synthesis. For structured tool sources, including both API groups and MCP skills, we use a second branch to teach execution and planning directly from executable interfaces. We first build a tool inventory by grouping related endpoints or skills, parsing tool definitions, and removing low-signal sources with lightweight prescreening. We then use GPT-5 to assign each source a quality score together with a feasibility profile over trajectory families, including simple single-call use, complex multiple/parallel tool use, and information-missing settings. For sources that pass the

Table 2 | Data mixture statistics. Token counts are in billions. Slash-separated values denote source corpus / context-grounded augmentation.

Source	Tokens (B)	Samples	Ratio
Web	4.4 / 4.1	6.86M	42%
PDF	2.6 / 2.1	1.34M	23%
Code	3.8 / 1.5	2.60M	26%
Native Agentic Trajectory	1.8	0.42M	9%
Total	20.3	11.22M	100%

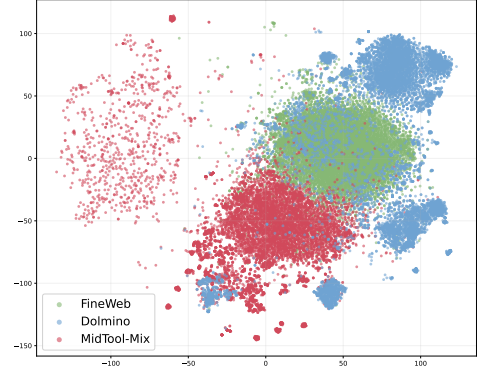


Figure 3 | t-SNE visualization.

quality filter, we recover developer-oriented documentation context, normalize the available tools into canonical executable schemas, and apply targeted schema refinement only when argument descriptions are underspecified. We next synthesize diverse user personas and let the model propose candidate trajectory plans, but the final allocation is enforced by a deterministic quality-adaptive budget controller that conditions on source quality, tool count, argument structure, and feasibility constraints, with preference for multi-turn trajectories. This prevents overproducing trivial samples and shifts generation toward richer multi-turn behaviors for high-capability sources. Finally, category-specific generators instantiate the planned trajectories, using a mixture of GPT-5, GPT-5.1, and GPT-5.2 for generation. The resulting trajectories are strictly validated for turn ordering, schema grounding, required arguments, and tool-response consistency; invalid generations are additionally retried with quality-control feedback before discard. We also mix in rollout trajectories collected from the agentic world model (AWM) (Wang et al., 2026) synthesized environments, so that the model learns not only isolated atomic capabilities, but also their composition into robust agent behavior. Finally, we also incorporate filtered agentic traces from the Nemotron Agentic dataset (Blakeman et al., 2025) to further scale up the native trajectory portion.

2.4. Analysis of MidTool-Mix

MidTool-Mix. Our final training mixture consists of three components: (1) a high-quality source mixture spanning filtered web, code, and PDF corpora; (2) context-grounded trajectories, including QAs, derived from those sources; and (3) native agentic trajectories, including synthesized trajectories built from real corpora, collected interaction rollouts, and filtered Nemotron Agentic dataset. Following Olmo et al. (2025), we normalize all trajectories into a plain chat-style template without special control tokens such as `<im_start>`.

Table 2 summarizes the resulting composition. In total, MidTool-Mix contains 20.3B tokens and 11.22M samples. For web, PDF, and code, we report the retained source tokens and the associated context-grounded augmentation separately using slash-separated counts. The overall mixture is intentionally balanced across web (42%), code (26%), and PDF (23%) sources, while native agentic trajectories contribute an additional 9% of the total budget. This allocation preserves broad technical coverage for grounding from natural documents, while reserving a non-trivial fraction of the mixture for executable agent supervision. Appendix A.4 provides a finer-grained breakdown by augmentation type, trajectory shape, and tool inventory.

Figure 3 provides a qualitative comparison against baseline pre-training dataset FineWeb and mid-training mix Dolma 3 Dolmino (Dolmino) by Olmo et al. (2025). The visualization comes from 2K sampled examples per dataset using the embedding model Arctic-Embed-2.0-L (Yu et al., 2024).

MidTool-Mix marginally overlaps with both reference corpora, which is expected because our work still draws on broad technical web data. Regarding the distinct region of Dolmino, we believe it is associated with the effort on math/science reasoning-focused corpus in Dolmino, which is not our primary focus. At the same time, MidTool-Mix occupies distinct regions that are not covered well by either dataset, indicating that our pipeline does not merely recover a narrow subset of generic pretraining and dedicated mid-training data. Instead, it shifts the distribution toward documentation-heavy, workflow-oriented, and agentic tool-use content. We additionally audit MidTool-Mix for overlap with our evaluation benchmarks and find no evidence of leakage (Appendix A.3).

3. Experiment

3.1. Setup

Baselines and Training Setup. Unlike prior work that studies tool use primarily as a post-training objective, our pipeline starts from pretrained base models and injects tool-use capability earlier through mid-training. We study two base models, Qwen3-4B-Base and Qwen3-8B-Base, and compare four training recipes for each scale: raw base model with SFT and optional RL; base model mid-trained with MidTool-Mix, followed by SFT and optional RL. For SFT, we fine-tune the model on a sampled 100K tool-use subset from TOUCAN (Xu et al., 2025). Both mid-training and SFT are conducted with ArcticTraining (Snowflake, 2026) on 32 H200 GPUs. We adopt the AWM setup (Wang et al., 2026) using 526 synthetic tool-use environments for agentic RL training with 8 B200 GPUs. Detailed hyperparameters are deferred to Appendix B. We also compare to the post-trained Qwen3-4B and Qwen3-8B models, and disable thinking to align with our setting.

Benchmarks. We evaluate general agentic tool-use capability on three complementary benchmarks: v3 split of BFCL (BFCLv3) (Patil et al., 2025), verified τ^2 -Bench (Barres et al., 2025; Cuadron et al., 2025), and MCP-Universe (Luo et al., 2025). BFCL measures function-calling quality across both single-turn and multi-turn settings, and is particularly useful for isolating grounded tool selection, argument construction, and hallucination behavior. τ^2 -Bench stresses interactive task completion in realistic verticals such as airline, retail, and telecom, making it a useful probe of multi-step execution and recovery. MCP-Universe evaluates execution over real MCP servers spanning domains such as browser automation, finance, location, and web search, and therefore serves as a harder test of out-of-distribution tool generalization. We follow the eval harness developed by Wang et al. (2026).

3.2. Main Results

Tables 3, 4, and 5 summarize the main results. Across both model sizes, we observe a consistent pattern: MidTool-Mix improves downstream agentic tool-use performance over SFT-only baselines, and RL usually compounds these gains further. The largest improvements typically appear on benchmarks or subsets that require longer interaction horizons, stronger schema grounding, or more robust adaptation to unfamiliar tools. Additional training-dynamics evidence is provided in Appendix C, where we show that MidTool-Mix also yields better SFT convergence and faster early-stage RL adaptation. Appendix C.3 also reports an exploratory pilot on visual tool use.

Mid-training yields capabilities beyond post-training alone. As shown in Table 3, adding MidTool-Mix before SFT substantially improves overall BFCL at both 4B and 8B scale, with the strongest variants appearing after RL. The gains are especially pronounced on the multi-turn subsets: at 4B, the average multi-turn score rises by more than 10 points over the SFT-only baseline, and the same pattern holds at 8B, where MidTool-Mix combined with RL yields the best multi-turn performance. The fact that the largest gains concentrate on harder multi-step settings suggests that mid-training contributes

Table 3 | BFCLv3 results for Qwen3-4B and Qwen3-8B base models. For each scale, we compare SFT and SFT+RL with and without prior mid-training on MidTool-Mix; released Qwen3 models are included as references. We report single-turn, multi-turn (MF = Missing Function, MP = Missing Parameters, LC = Long Context), hallucination (Hallu.), and overall performance. All values are percentages and higher is better.

Model Setting	Single Turn		Multi-Turn					Hallu.	Overall
	Non-live	Live	Base	MF	MP	LC	Avg.		
Qwen3-4B	39.58%	24.35%	10.50%	9.00%	6.00%	10.00%	8.88%	9.05%	24.27%
Qwen3-4B-Base + SFT	59.94%	43.75%	21.50%	14.00%	10.00%	16.50%	15.50%	60.46%	39.73%
Qwen3-4B-Base + SFT + RL	59.65%	39.90%	27.00%	18.00%	13.00%	18.00%	19.00%	55.19%	39.51%
Qwen3-4B-Base + MidTool-Mix + SFT	66.38%	57.74%	36.00%	23.00%	19.50%	28.00%	26.63%	56.95%	50.25%
Qwen3-4B-Base + MidTool-Mix + SFT + RL	76.44%	58.48%	34.50%	24.50%	20.50%	31.00%	27.63%	60.10%	54.18%
Qwen3-8B	30.73%	26.13%	31.00%	22.50%	15.00%	21.50%	22.50%	9.77%	26.45%
Qwen3-8B-Base + SFT	66.40%	51.22%	30.50%	26.50%	16.00%	28.00%	25.25%	65.03%	47.62%
Qwen3-8B-Base + SFT + RL	68.52%	39.60%	43.00%	22.00%	19.00%	33.00%	29.25%	52.79%	45.79%
Qwen3-8B-Base + MidTool-Mix + SFT	65.73%	55.37%	38.00%	31.50%	24.00%	35.50%	32.25%	59.82%	51.12%
Qwen3-8B-Base + MidTool-Mix + SFT + RL	72.58%	55.14%	50.50%	33.50%	25.00%	41.50%	37.63%	64.22%	55.12%

Table 4 | τ^2 -Bench results for Qwen3-4B and Qwen3-8B base models. For each scale, we compare SFT and SFT+RL with and without prior mid-training on MidTool-Mix; released Qwen3 models are included as references. We report Pass@1 and Pass@4 for airline, retail, telecom, and overall; higher score is stronger interactive capacity.

Model Setting	Airline		Retail		Telecom		Overall	
	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4
Qwen3-4B	21.00%	32.00%	16.23%	35.09%	3.51%	5.26%	11.87%	22.30%
Qwen3-4B-Base + SFT	17.50%	34.00%	9.43%	24.56%	3.73%	10.53%	8.54%	20.50%
Qwen3-4B-Base + SFT + RL	31.00%	52.00%	18.20%	39.47%	0.00%	0.00%	13.04%	25.54%
Qwen3-4B-Base + MidTool-Mix + SFT	17.00%	38.00%	20.83%	47.37%	1.54%	4.39%	12.23%	28.06%
Qwen3-4B-Base + MidTool-Mix + SFT + RL	20.00%	40.00%	33.55%	57.89%	6.36%	18.42%	19.96%	38.49%
Qwen3-8B	19.00%	44.00%	12.06%	37.72%	5.04%	16.67%	10.43%	30.22%
Qwen3-8B-Base + SFT	12.00%	38.00%	15.79%	38.60%	4.39%	13.16%	10.43%	28.06%
Qwen3-8B-Base + SFT + RL	15.50%	38.00%	30.92%	57.02%	5.26%	19.30%	17.63%	38.13%
Qwen3-8B-Base + MidTool-Mix + SFT	18.50%	46.00%	26.32%	61.40%	1.54%	3.51%	14.75%	34.89%
Qwen3-8B-Base + MidTool-Mix + SFT + RL	24.50%	52.00%	39.04%	67.54%	2.19%	6.14%	21.31%	39.57%

capabilities that are not reliably induced by standard post-training alone, especially stronger grounding of atomic agentic capabilities together with better planning and execution over longer interaction horizons.

The learned gains transfer to realistic agentic tasks. We see this most clearly on τ^2 -Bench and MCP-Universe in Table 4 and 5, which are both more realistic and complex than BFCL. On τ^2 -Bench, MidTool-Mix substantially improves overall performance at both scales, nearly doubling overall Pass@1 at 4B and yielding clear gains in overall Pass@1 / Pass@4 at 8B. The gains are strongest on airline and retail, while telecom remains difficult, but the overall pattern still shows transfer to interactive task completion. The same trend appears on MCP-Universe, where MidTool-Mix raises both overall score and pass rate at 4B and 8B, with especially visible gains after RL. Taken together, these two benchmarks suggest that MidTool-Mix does not merely improve tool use, but learns a more general prior for interacting with realistic environments and previously unseen tool ecosystems.

General tool-use mid-training reveals a meaningful capability boundary. As shown in Table 5, although MidTool-Mix substantially improves the overall score, the web search subset stays at 0.00.

Table 5 | MCP-Universe results for Qwen3-4B and Qwen3-8B base models. For each scale, we compare SFT and SFT+RL with and without prior mid-training on MidTool-Mix; released Qwen3 models are included as references. We report score and pass rate; higher is better.

Model Setting	Browser Auto.		Financial		Location		Multi-server		Web Search		Overall	
	Score	Pass	Score	Pass	Score	Pass	Score	Pass	Score	Pass	Score	Pass
Qwen3-4B	19.42	5.88%	9.17	7.50%	43.51	2.86%	16.17	0.00%	0.00	0.00%	16.05	3.35%
Qwen3-4B-Base + SFT	15.44	2.94%	5.00	5.00%	38.98	0.00%	13.67	0.00%	0.00	0.00%	13.20	1.68%
Qwen3-4B-Base + SFT + RL	17.65	5.88%	6.67	5.00%	39.69	0.00%	17.00	0.00%	0.00	0.00%	14.50	2.23%
Qwen3-4B-Base + MidTool-Mix + SFT	21.81	8.82%	14.27	12.50%	46.81	2.86%	19.50	0.00%	0.00	0.00%	18.66	5.03%
Qwen3-4B-Base + MidTool-Mix + SFT + RL	19.73	8.82%	38.33	37.50%	49.50	0.00%	16.17	0.00%	0.00	0.00%	23.80	10.06%
Qwen3-8B	22.92	8.82%	12.50	12.50%	25.78	0.00%	7.83	0.00%	0.00	0.00%	13.06	4.47%
Qwen3-8B-Base + SFT	17.40	5.88%	7.50	7.50%	40.52	0.00%	20.33	5.00%	0.00	0.00%	15.18	3.35%
Qwen3-8B-Base + SFT + RL	13.60	2.94%	17.50	17.50%	36.26	2.86%	18.67	0.00%	0.00	0.00%	15.67	5.03%
Qwen3-8B-Base + MidTool-Mix + SFT	19.12	5.88%	11.46	10.00%	48.31	2.86%	19.50	0.00%	0.00	0.00%	17.82	3.91%
Qwen3-8B-Base + MidTool-Mix + SFT + RL	24.63	11.76%	30.83	30.00%	58.35	2.86%	19.50	0.00%	0.00	0.00%	25.16	9.50%

Table 6 | Ablation across BFCLv3, τ^2 -Bench, and MCP-Universe under the Qwen3-4B-Base + SFT setting, where the downstream post-training recipe is fixed and only the mid-training corpus is changed. Rows under MidTool are additive over the filtered raw sources: *Processed data w/o traj.* mid-trains on the processed raw data only, with no synthesized trajectories; the next two rows each add a *single* synthesis branch on top of it, and are therefore alternatives rather than successive steps; MidTool-Mix is the complete mixture combining all. Dolmino-20BT is a matched-budget generic mid-training baseline.

Mid-training Data	BFCLv3				τ^2 -Bench		MCP-Universe	
	Non-live	Live	Multi-turn	Overall	Pass@1	Pass@4	Score	Pass
No Mid-training	59.94% _{+0.0}	43.75% _{+0.0}	15.50% _{+0.0}	39.73% _{+0.0}	8.54% _{+0.0}	20.50% _{+0.0}	13.20 _{+0.0}	1.68% _{+0.0}
Dolmino-20BT	61.44% _{+1.5}	51.74% _{+8.0}	16.13% _{+0.6}	43.10% _{+3.4}	7.37% _{-1.2}	21.22% _{+0.7}	5.41 _{-7.8}	0.00% _{-1.68}
MidTool								
Processed data w/o traj.	60.40% _{+0.5}	52.60% _{+8.9}	13.90% _{-1.6}	42.30% _{+2.6}	7.30% _{-1.2}	21.90% _{+1.4}	12.20 _{-1.0}	3.03% _{+1.4}
+ native agentic traj.	68.21% _{+8.3}	55.81% _{+12.1}	18.75% _{+3.3}	47.59% _{+7.9}	4.23% _{-4.3}	12.95% _{-7.6}	6.80 _{-6.4}	1.12% _{-0.6}
+ context grounded traj.	62.73% _{+2.8}	50.26% _{+6.5}	21.00% _{+5.5}	44.66% _{+4.9}	8.99% _{+0.5}	21.94% _{+1.4}	8.46 _{-4.7}	1.12% _{-0.6}
MidTool-Mix	66.38%_{+6.4}	57.74%_{+14.0}	26.63%_{+11.1}	50.25%_{+10.5}	12.23%_{+3.7}	28.06%_{+7.6}	18.66_{+5.5}	5.03%_{+3.4}

This contrast is informative rather than merely negative: other MCP domains, especially browser automation, financial analysis, and location, do improve meaningfully, so the issue is not a failure to transfer to MCP tools. Instead, it isolates a distinct class of agentic behavior, namely deep-search-style tasks that require longer-horizon evidence gathering, iterative refinement, and stronger agent-level control flow. This is an important empirical signal about the structure of agentic capability: broad tool-use supervision teaches reusable priors for schema grounding, tool selection, and interaction with unfamiliar APIs, while highly exploratory domains appear to require dedicated trajectory data and training objectives. In this sense, MidTool-Mix helps map the boundary between general tool use and specialized agency, providing guidance for future mid-training work on search-heavy and other specialized settings.

3.3. Ablation Study

To study the effect of data design, we fix the 4B SFT recipe and vary only the mid-training corpus. We decompose MidTool-Mix additively: the processed raw sources alone, each synthesis branch added on top of them, and the complete mixture. We compare against a matched-budget generic mid-training baseline, Dolmino-20BT (Olmo et al., 2025), with no-mid-training setting as a reference.

The two MidTool subsets play different but complementary roles. The processed raw sources alone already provide a positive standalone signal: with no synthesized data at all, they improve BFCLv3 overall by +2.6 and MCP-Universe Pass by +1.4 over no mid-training, and stay competitive with Dolmino-20BT on BFCL overall and τ^2 -Bench Pass@4 while transferring substantially better to MCP-Universe. We note that Dolmino-20BT is itself not a purely non-synthetic baseline, as the released mixture also contains model-generated components. The complete mixture substantially improves over no mid-training, and even the variant that adds only context-grounded augmentation remains well above both no mid-training and Dolmino-20BT. Compared with Dolmino-20BT, this variant is only marginally weaker on the live subset, but stronger on non-live, multi-turn, and overall. On τ^2 -Bench, it is similar: the context-grounded-only variant exceeds both no mid-training and Dolmino-20BT. Notably, the native agentic trajectory subset is the only subset that contains trajectories synthesized by proprietary models. The fact that this variant still substantially outperforms no mid-training on BFCLv3 and τ^2 -Bench, and exceeds Dolmino-20BT on nearly all metrics, indicates that the gains of MidTool-Mix are not solely driven by proprietary-model-synthesized trajectories. This branch is nonetheless still model-synthesized, with an open-weight teacher; the strictly non-distilled contribution is isolated by the filtered data split above.

The two branches also contribute asymmetrically. Adding native agentic trajectories alone yields the larger BFCLv3 gain (+7.9 vs. +4.9 overall), indicating that executable trajectories are especially important for precise function calling. Adding context-grounded augmentation alone is the stronger of the two on τ^2 -Bench and MCP-Universe, indicating that grounding-oriented supervision is especially important for transfer. On BFCL non-live the native-only variant is even slightly ahead of the complete mixture (68.21% vs. 66.38%), but it gives up large margins on multi-turn, τ^2 -Bench, and MCP-Universe; combining both branches is the only configuration that improves over no mid-training on all eight metrics. MCP-Universe makes the complementarity especially clear: both single-branch variants fall below no mid-training, yet still outperform Dolmino-20BT, indicating that each subset contributes a meaningful but incomplete agentic prior. More broadly, the relatively competitive BFCL result of Dolmino-20BT, contrasted with its much weaker transfer on τ^2 -Bench and MCP-Universe, suggests that generic instruction-following-style mid-training may help simple function-calling behavior, but transfers poorly to more agentic settings.

4. Related Work

Mid-training for Agentic Capability The advancing agentic capability of frontier models emphasizes that strong agentic behavior is not obtained from post-training solely, but from combining a strong pretrained base with additional large-scale capability shaping for long-horizon mid-training (Moonshot AI, 2026; Xiao et al., 2026). For example, the GLM team describes its efforts in mid-training on reasoning and agentic data for long-context settings and on repo-level code data for software engineering tasks (Z.ai, 2026; Zeng et al., 2025). Tongyi DeepResearch proposes agentic mid-training with a scaled agentic data pipeline for deep-research foundations (Su et al., 2025; Team et al., 2025b). Yang et al. (2025) and Zeng et al. (2026) propose SWE-centric pipelines for agentic mid-training data. While Olmo et al. (2025) discloses a mid-training recipe, it primarily focuses on high-quality data in general domains, with a mixture that emphasizes reasoning tasks such as math and science. Different from prior work (Wang et al., 2025), we construct an open-source mid-training corpus for agentic tool use through a dedicated pipeline.

Post-training Dataset for Tool Use Early work on tool-use data focuses on construction from tool definitions or model self-annotation. Toolformer (Schick et al., 2023) shows that language models can insert API calls into text with self-supervision, while ToolLLM (Qin et al., 2023) and APIGen

(Liu et al., 2024) expand this line to large collections of real APIs and verifiable function-calling instances. Later work moves from single-turn invocation to richer agent interaction trajectories. AgentBank (Song et al., 2024) collects diverse interaction traces for general agent fine-tuning, and ToolACE (Liu et al., 2025) shows that large-scale, carefully curated function-calling supervision remains highly effective for post-training tool-use alignment. More recent work emphasizes multi-turn, environment-grounded data synthesis at larger scale. APIGen-MT (Prabhakar et al., 2025) generates multi-turn trajectories through simulated agent-human interaction, TOUCAN (Xu et al., 2025) synthesizes 1.5M tool-agentic examples from real-world MCP environments, and Simia (Li et al., 2025) focuses on synthetic environments and further explores simulator-backed training data for interactive agents. These efforts substantially improve the availability of post-training supervision for tool calling and agent interaction. Our focus is complementary and parallel to this line of work: we study an open-source corpus and data pipeline for agentic *mid-training*, which can provide a broader prior for tool-use behavior and in turn work synergistically with downstream post-training datasets and objectives.

5. Conclusion

In this work, we study whether general tool use benefits from dedicated mid-training rather than being left entirely to post-training. We introduce MidTool, a scalable corpus-construction pipeline, and MidTool-Mix, a 20.3B-token mixture that pairs filtered source corpora with context-grounded augmentation and native agentic trajectories. Across 4B and 8B base models, mid-training on MidTool-Mix consistently improves downstream performance on BFCL, τ^2 -Bench, and MCP-Universe under both SFT and RL, with especially strong gains on multi-turn and interactive settings. These gains are complementary to, rather than a substitute for, stronger post-training: the downstream recipe is held fixed throughout our experiments, and we view scaling post-training supervision and strengthening tool-use mid-training as two axes that should advance together.

Ablations show that these improvements stem from the structure of the mixture: removing either synthesis branch degrades performance, indicating that grounding-oriented and execution-oriented supervision are complementary. Our results also surface a meaningful capability boundary: while MidTool-Mix improves broad transfer to unseen tools and MCP domains, it contributes marginally to deep-search-style exploratory behaviors. This suggests that general tool-use mid-training provides reusable priors for schema grounding and workflow composition, whereas highly exploratory domains require dedicated trajectory data. Future work includes scaling native trajectory collection, broadening tool ecosystem coverage, and constructing specialized mid-training mixtures for domain-specific agentic behaviors.

References

- V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- A. Blakeman, A. Grattafiori, A. Basant, A. Gupta, A. Khattar, A. Renduchintala, A. Vavre, A. Shukla, A. Bercovich, A. Ficek, et al. Nemotron 3 nano: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. *arXiv preprint arXiv:2512.20848*, 2025.
- A. Cuadron, P. Yu, Y. Liu, and A. Gupta. Saber: Small actions, big errors—safeguarding mutating steps in llm agents. *arXiv preprint arXiv:2512.07850*, 2025.

- GH Archive. Gh archive: Public github event data. <https://www.gharchive.org/>, 2026. Public archive of GitHub timeline events.
- X. Guo, U. Tyagi, A. Gosai, P. Vergara, J. Park, E. G. H. Montoya, C. B. C. Zhang, B. Hu, Y. He, B. Liu, et al. Beyond seeing: Evaluating multimodal llms on tool-enabled image perception, transformation, and reasoning. *arXiv preprint arXiv:2510.12712*, 2025.
- B. Isik, N. Ponomareva, H. Hazimeh, D. Paparas, S. Vassilvitskii, and S. Koyejo. Scaling laws for downstream task performance in machine translation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=vPOMTkmSiu>.
- A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov. Bag of tricks for efficient text classification. In *Proceedings of the 15th conference of the European chapter of the association for computational linguistics: volume 2, short papers*, pages 427–431, 2017.
- H. Kydlíček, G. Penedo, and L. von Werra. Finepdfs. <https://huggingface.co/datasets/HuggingFaceFW/finepdfs>, 2025.
- R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Y. Li, H. A. Inan, X. Yue, W.-N. Chen, L. Wutschitz, J. Kulkarni, R. Poovendran, R. Sim, and S. Rajmohan. Simulating environments with reasoning models for agent training. *arXiv preprint arXiv:2511.01824*, 2025.
- H. Liu, S. M. Xie, Z. Li, and T. Ma. Same pre-training loss, better downstream: Implicit bias matters for language models. In *International Conference on Machine Learning*, pages 22188–22214. PMLR, 2023.
- W. Liu, X. Huang, X. Zeng, X. Hao, S. Yu, D. Li, S. Wang, W. Gan, Z. Liu, Y. Yu, Z. Wang, Y. Wang, W. Ning, Y. Hou, B. Wang, C. Wu, X. Wang, Y. Liu, Y. Wang, D. Tang, D. Tu, L. Shang, X. Jiang, R. Tang, D. Lian, Q. Liu, and E. Chen. Toolace: Winning the points of llm function calling, 2025. URL <https://arxiv.org/abs/2409.00920>.
- Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482, 2024.
- Z. Luo, Z. Shen, W. Yang, Z. Zhao, P. Jwalapuram, A. Saha, D. Sahoo, S. Savarese, C. Xiong, and J. Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*, 2025.
- Moonshot AI. Kimi k2.5: Native multimodal agentic model, 2026. URL <https://github.com/MoonshotAI/Kimi-K2.5>. Official repository and technical report for Kimi K2.5.
- T. Olmo, :, A. Ettinger, A. Bertsch, B. Kuehl, D. Graham, D. Heineman, D. Groeneveld, F. Brahman, F. Timbers, H. Ivison, J. Morrison, J. Poznanski, K. Lo, L. Soldaini, M. Jordan, M. Chen, M. Noukhovitch, N. Lambert, P. Walsh, P. Dasigi, R. Berry, S. Malik, S. Shah, S. Geng, S. Arora, S. Gupta, T. Anderson, T. Xiao, T. Murray, T. Romero, V. Graf, A. Asai, A. Bhagia, A. Wettig, A. Liu, A. Rangapur, C. Anastasiades, C. Huang, D. Schwenk, H. Trivedi, I. Magnusson, J. Lochner, J. Liu, L. J. V. Miranda, M. Sap, M. Morgan, M. Schmitz, M. Guerquin, M. Wilson, R. Huff, R. L. Bras, R. Xin, R. Shao, S. Skjonsberg, S. Z. Shen, S. S. Li, T. Wilde, V. Pyatkin, W. Merrill, Y. Chang, Y. Gu, Z. Zeng, A. Sabharwal, L. Zettlemoyer, P. W. Koh, A. Farhadi, N. A. Smith, and H. Hajishirzi. Olmo 3, 2025. URL <https://arxiv.org/abs/2512.13961>.

- S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>.
- G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. V. Werra, and T. Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=n6SCkn2QaG>.
- A. Prabhakar, Z. Liu, M. Zhu, J. Zhang, T. Awalgaonkar, S. Wang, Z. Liu, H. Chen, T. Hoang, J. C. Niebles, et al. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. *arXiv preprint arXiv:2504.03601*, 2025.
- Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Snowflake. Arctictraining: Simplifying and accelerating post-training for llms. <https://github.com/snowflakedb/ArcticTraining>, 2026. GitHub repository, accessed 2026-03-31.
- Y. Song, W. Xiong, X. Zhao, D. Zhu, W. Wu, K. Wang, C. Li, W. Peng, and S. Li. Agentbank: Towards generalized llm agents via fine-tuning on 50000+ interaction trajectories. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 2124–2141, 2024.
- L. Su, Z. Zhang, G. Li, Z. Chen, C. Wang, M. Song, X. Wang, K. Li, J. Wu, X. Chen, Z. Qiao, Z. Zhang, H. Yin, S. Cai, R. Fang, Z. Tao, W. Yin, C. Qian, Y. Jiang, P. Xie, F. Huang, and J. Zhou. Scaling agents via continual pre-training, 2025. URL <https://arxiv.org/abs/2509.13310>.
- M. L. Team, A. Gui, B. Li, B. Tao, B. Zhou, B. Chen, C. Zhang, C. Han, C. Yang, C. Zhang, et al. Introducing longcat-flash-thinking: A technical report. *arXiv preprint arXiv:2509.18883*, 2025a.
- T. D. Team, B. Li, B. Zhang, D. Zhang, F. Huang, G. Li, G. Chen, H. Yin, J. Wu, J. Zhou, et al. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*, 2025b.
- C. Tu, X. Zhang, R. Weng, R. Li, C. Zhang, Y. Bai, H. Yan, J. Wang, and X. Cai. A survey on llm mid-training. *arXiv preprint arXiv:2510.23081*, 2025.
- Z. Wang, F. Zhou, X. Li, and P. Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling, 2025. URL <https://arxiv.org/abs/2506.20512>.
- Z. Wang, C. Xu, B. Liu, Y. Wang, S. Han, Z. Yao, H. Yao, and Y. He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026.
- M. Weber, D. Fu, Q. Anthony, Y. Oren, S. Adams, A. Alexandrov, X. Lyu, H. Nguyen, X. Yao, V. Adams, et al. Redpajama: an open dataset for training large language models. *Advances in neural information processing systems*, 37:116462–116492, 2024.
- L. Wiedmann, O. Zohar, A. Mahla, X. Wang, R. Li, T. Frere, L. von Werra, A. R. Gosthipaty, and A. Marafioti. Finevision: Open data is all you need. *arXiv preprint arXiv:2510.17269*, 2025.

- B. Xiao, B. Xia, B. Yang, B. Gao, B. Shen, C. Zhang, C. He, C. Lou, F. Luo, G. Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- Z. Xu, A. M. Soria, S. Tan, A. Roy, A. S. Agrawal, R. Poovendran, and R. Panda. Toucan: Synthesizing 1.5m tool-agentic data from real-world mcp environments, 2025. URL <https://arxiv.org/abs/2510.01179>.
- Z. Yang, S. Wang, K. Fu, W. He, W. Xiong, Y. Liu, Y. Miao, B. Gao, Y. Wang, Y. Ma, Y. Li, Y. Liu, Z. Hu, K. Zhang, S. Wang, H. Chen, F. Sung, Y. Liu, Y. Gao, Z. Yang, and T. Liu. Kimi-dev: Agentless training as skill prior for swe-agents, 2025. URL <https://arxiv.org/abs/2509.23045>.
- P. Yu, L. Merrick, G. Nuti, and D. Campos. Arctic-embed 2.0: Multilingual retrieval without compromise, 2024. URL <https://arxiv.org/abs/2412.04506>.
- Z.ai. Glm-5, 2026. URL <https://docs.z.ai/guides/llm/glm-5>. Official model documentation describing the GLM-5 release, larger-scale pretraining, and asynchronous agent reinforcement learning.
- A. Zeng, X. Lv, Q. Zheng, Z. Hou, B. Chen, C. Xie, C. Wang, D. Yin, H. Zeng, J. Zhang, et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- J. Zeng, D. Fu, T. Mi, Y. Zhuang, Y. Huang, X. Li, L. Ye, M. Xie, Q. Hua, Z. Huang, M. Jiang, H. Wang, J. Lin, Y. Xiao, J. Sun, Y. Wu, and P. Liu. davinci-dev: Agent-native mid-training for software engineering, 2026. URL <https://arxiv.org/abs/2601.18418>.

A. More Details for MidToolPipeline

A.1. Benchmark Exclusion During Data Collection

To reduce benchmark leakage from the GitHub repository slice, we explicitly exclude known benchmark and evaluation repositories before repository crawling and preprocessing. In practice, our code-data pipeline loads a blacklist of repository names and skips matched repositories entirely during dataset construction. The blacklist includes repositories associated with BFCL-related benchmarks, τ -Bench / τ^2 -Bench, MCP-Universe and related MCP evaluation suites, together with other widely used agentic and tool-use benchmarks.

This blacklist-based exclusion is integrated into the data pipeline rather than applied as an ad hoc postprocessing step. We actively maintain and update this list as new public agentic benchmarks appear, so that future runs of the pipeline can continue to enforce the same benchmark-exclusion policy during data collection.

A.2. Web and PDF Filtering Pipeline

For web and PDF midtraining sources, we use a separate four-phase pipeline consisting of keyword/URL filtering, fastText classification, quality filtering, and MinHash-based deduplication.

Phase 1: High-recall prescreening. The first phase acts as a high-recall prescreener. It combines matches against a curated vocabulary of software-development terms with URL patterns associated with documentation and technical reference sites, and it gives additional weight to documents containing code-like structure. This phase is intended to retain broadly relevant technical content, such as API references, SDK or library documentation, CLI-style help pages, platform documentation, troubleshooting pages, and developer Q&A, while removing most general web noise.

Phase 2: fastText classification. The second phase is a lightweight fastText (Joulin et al., 2017) classifier trained from LLM-labeled seed data. To build the seed set, we sample from 1M web documents and 3M PDF documents. We then use Qwen2.5-7B-Instruct to annotate the seed set. Our labeling prompt defines positive examples, including documentation, tutorials, configuration, debugging traces, and code-centered technical discussion, and negatives as non-technical or low-value material for tool-use purposes. These labeled examples are then converted into positive and negative training sets for fastText. We apply stricter classification thresholds for PDF data because PDF extraction is noisier and more heterogeneous than web text.

Phase 3: Quality filtering. The third phase applies document-level quality controls. For both web and PDF data, we filter on language confidence, length, word count, symbol density, and estimated code ratio. The goal is to remove low-quality extraction artifacts, extremely short or noisy documents, and documents dominated by uninterpretable symbols or raw code dumps. For PDF data, we additionally apply an OCR-quality filter to address extraction noise.

Phase 4: Deduplication. Finally, we use MinHash LSH to reduce repeated content across sources and shards.

A.3. Post-hoc Contamination Analysis

The blacklist described above prevents benchmark repositories from entering the code slice, but it does not bound leakage that may arrive through web pages, PDFs, or model-synthesized supervision. We therefore additionally audit the finished mixture with DeCon (Olmo et al., 2025), using its default configuration and lowering the token-length threshold so that short benchmark items are also covered. We scan the web and PDF slices together with the teacher-synthesized data against all three evaluation benchmarks used in this paper: BFCLv3, τ^2 -Bench, and MCP-Universe.

DeCon flags fewer than 20 candidates in total, all of them from the web slice and all against BFCLv3, and none for τ^2 -Bench or MCP-Universe. Manual inspection confirms that every flagged item is a false positive: generic function-calling and API-documentation text that shares surface n -grams with benchmark items, but that contains no benchmark instances and no reference answers. We therefore find no evidence of actual benchmark leakage into MidTool-Mix.

Since DeCon operates by surface-level n -gram matching, this analysis bounds verbatim overlap rather than semantic or schema-level similarity, which is particularly relevant for MCP-related tool definitions. We leave a deeper semantic audit of such overlap to future work.

A.4. Composition Analysis of MidTool-Mix

Table 2 in the main text reports the mixture at the level of source families. Here we provide a finer-grained view of what each slice actually contains.

Augmentation type per source. A source-level breakdown alone hides how much of each slice carries synthesized supervision. Decomposing every web/pdf/code sample into one of {source-only, +QA, +QA + trajectory} gives Table 7. The three slices play visibly different roles: web is dominated by QA-style augmentation, PDF contributes the largest share of full trajectories, and code remains mostly raw source, reflecting that much of its tool-use signal is already present in the original files.

Table 7 | Augmentation type per source slice, as a fraction of samples within that slice. Sample counts match the *Samples* column of Table 2.

Slice	source-only	+QA	+QA + traj.
web (6.86M)	36.4%	52.2%	11.3%
pdf (1.34M)	30.8%	13.4%	55.7%
code (2.60M)	68.9%	4.2%	26.8%

Trajectory shape. Among samples that contain agentic trajectories, Table 8 reports per-document statistics, where a turn is a user message, a step is an assistant message, and a tool call is a single tool invocation. Context-grounded trajectories derived from documents tend to have more assistant steps and tool calls per user turn, whereas native trajectories synthesized from executable environments are longer in user turns.

Tool inventory. MidTool-Mix exposes 2.60M unique tool names across its tool calls. Mapping names to functional categories with keyword rules gives Table 9. Beyond the head categories, 37.2% of all calls fall into a domain-specific long tail, which quantifies the tool diversity of the mixture and distinguishes it from prior tool-use post-training corpora that are typically built over a fixed tool inventory.

Table 8 | Per-document trajectory statistics among samples containing agentic trajectories. A turn is a user message, a step is an assistant message, and a tool call is a single tool invocation.

Metric	Stat	web	pdf	code	native
Turns	mean	2.8	3.0	3.1	4.0
	median	3	3	3	4
	max	15	17	8	69
Steps	mean	5.3	7.1	6.0	3.8
	median	5	6	5	4
	max	54	77	58	21
Tool calls	mean	3.1	4.8	3.3	2.4
	median	2	4	2	2
	max	72	78	103	66

Table 9 | Functional categories of tool calls in MidTool-Mix, obtained by keyword mapping over tool names.

Category	Calls	Share	Examples
Code execution / Shell / DevOps	862K	10.9%	run_command, kubectl_get
List / Get (generic)	811K	10.2%	list_tools, get_*
File / Filesystem	380K	4.8%	read_file, list_dir
Set / Update / Modify	354K	4.5%	set_*, configure
Add / Create	348K	4.4%	create_*, register_*
Search / Lookup	335K	4.2%	search, lookup
AI / NLP / ML	249K	3.1%	BertFor*, embed
Web / HTTP / API	243K	3.1%	http_get, fetch_url
Database / SQL	201K	2.5%	sql_query, select_*
Hardware / Sensor / Device	153K	1.9%	read_register, modbus_*
13 other categories	920K	11.6%	Maps, Auth, Email, Finance, E-commerce
Domain-specific long tail	2.96M	37.2%	formNavigator, MemcachedSet

B. Experimental Details

We summarize the main training hyperparameters used in our experiments here. As described in the main text, both mid-training and SFT are conducted with ArcticTraining on 32 H200 GPUs, while RL follows the AWM setup with VeRL-style GRPO training on 8 B200 GPUs. Dataset composition details is described in the main text. The critical hyperparameters for mid-training, SFT, and RL are listed in Table 10, Table 11, and Table 12, respectively.

C. More Experimental Analysis

C.1. SFT Convergence Analysis

We further analyze the SFT training process using the same comparative-loss perspective as prior work on agentic continual pre-training (Su et al., 2025). Here, we ask whether MidTool-Mix produces a better initialization for downstream agentic post-training than either the raw base model or a generic continued-pretraining baseline.

We compare three 4B initializations under the same SFT setup and downstream corpus: Qwen3-4B-Base, Qwen3-4B-Base continued on Dolmino, and Qwen3-4B-Base continued on MidTool-Mix.

Table 10 | Mid-training hyperparameters for Qwen3-4B-Base and Qwen3-8B-Base.

Hyperparameter	Value
Epochs	1
Max Sequence length	8192
Optimizer	AdamW
Learning rate	3×10^{-5}
Betas	(0.9, 0.999)
Weight decay	0.01
LR schedule	WSD
Warmup steps	50
Target global token batch	4M tokens (with data packing)

Table 11 | Supervised fine-tuning hyperparameters.

Hyperparameter	Value
Max Sequence length	32768
Optimizer	AdamW
Learning rate	2×10^{-5}
Betas	(0.9, 0.95)
Weight decay	0.01
LR schedule	cosine, warmup ratio 0.001
Training batch size	128

Figure 4 reports the training loss trajectories over SFT steps, including both the raw per-step loss and a smoothed trend curve for readability. Because the downstream data, optimizer settings, and training recipe are held fixed, differences in convergence primarily reflect the quality of the initialization induced by the preceding mid-training stage.

Mid-training improves post-training optimization efficiency. Figure 4 shows the SFT training loss. Among the three initializations, MidTool-Mix enters SFT with the lowest loss, descends fastest in the early stage, and maintains the best loss throughout nearly the entire training run. This pattern matters because all three models are fine-tuned on the same downstream corpus with the same optimizer and schedule, so the gap is best explained by the quality of the initialization rather than by differences in the post-training recipe itself. We note, however, that SFT loss measures next-token prediction on the downstream corpus and is not itself a measure of tool-use capability: initializations that reach a similar loss can still differ substantially in schema grounding, tool selection, and multi-turn execution (Isik et al., 2025; Liu et al., 2023). We therefore read these curves as evidence about optimization behavior, and rely on the main-text benchmark results for capability claims.

The comparison against Dolmino is especially informative. Generic midtraining does improve over the raw base model, but it still converges more slowly and to a worse loss than MidTool-Mix. This suggests that the benefit is not merely a consequence of extra training tokens or additional compute. Instead, the agentic and tool-centered composition of MidTool-Mix appears to provide a better inductive bias for downstream tool-use supervision, making subsequent SFT and RL easier to optimize. This observation is consistent with the main-text results: the models that perform best after post-training are also the ones that begin SFT from a more favorable optimization landscape.

Table 12 | Reinforcement-learning hyperparameters.

Hyperparameter	Value
RL algorithm	GRPO
Total training steps	64
Learning rate	4B: 1×10^{-6} ; 8B: 5×10^{-7}
Data Batch size	32
PPO Mini-batch size	32
Rollouts per prompt	16
KL coefficient	0.001
Entropy coefficient	0.0
Clip ratio (high)	0.28
History limit	3
Maximum agent turns	20

C.2. RL Reward Analysis

We next examine the RL training dynamics to understand whether the gains from MidTool-Mix also persist during reinforcement learning. Here, the central question is not only the final reward reached in the training environment, but also how efficiently the model adapts during RL and whether that in-environment improvement translates to broader downstream generalization.

Mid-trained models adapt faster in RL, while their main advantage appears in out-of-environment generalization. Figure 5 shows a consistent pattern across both 4B and 8B. The MidTool-Mix models begin RL from clearly higher reward and improve much more rapidly in the early stage, whereas the corresponding base models start from substantially lower reward and require many more steps to reach a similar level. This suggests that mid-training provides a more favorable initialization not only for SFT, but also for subsequent policy optimization, allowing the model to adapt more efficiently once RL begins.

At the same time, the reward curves also show an important limitation of in-environment training reward as a proxy for final agent quality. By later RL steps, the two curves move much closer at both model sizes, which indicates that the raw base model can eventually learn much of the policy required for this specific RL environment. But this narrowing gap does not imply that the two training pipelines are equivalent. In the main results, the mid-trained models still achieve clearly stronger performance across BFCL, τ^2 -Bench, and MCP-Universe after RL, even when the in-training rewards become similar. Taken together, these results suggest that RL reward primarily reflects adaptation to the training environment, so its convergence does not imply that the two pipelines behave equivalently downstream. Our evidence for this is the persisting post-RL benchmark gap rather than the reward curves themselves. In other words, the main value of mid-training is not just faster reward acquisition during RL, but stronger generalizability after RL.

C.3. Pilot Study on Visual Tool Use

Inspired by the discussion of Kimi K2.5 (Moonshot AI, 2026), we also run a small pilot study on visual tool use as a zero visual tool cold-start transfer setting. This experiment is not the focus of our paper, rather we view it as an exploratory but potentially informative result: we simply extend text-only tool-use training and ask whether any capability transfers to a visual tool-use. Similar to our main setting, we mid-train a multimodal base model, gemma-3-4b-pt, on MidTool-Mix. For both the mid-trained model and the original base model, we then fine-tune it on 37.5k samples from

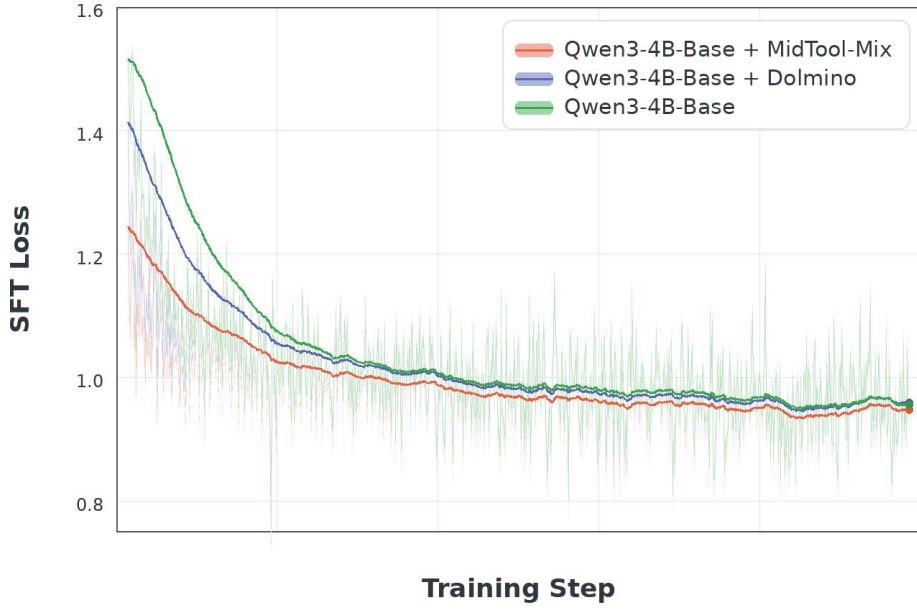


Figure 4 | SFT loss trajectories on the same downstream tool-use corpus for three Qwen3-4B initializations. MidTool-Mix starts from a lower loss, converges faster, and maintains the best loss throughout training compared with both the raw base model and the Dolmino mid-trained baseline.

FineVision (Wiedmann et al., 2025), followed by the same text-based tool SFT as in our main setting. For both mid-training and SFT, we do not include any visual tool-use data in training.

We evaluate on VisualToolBench (Guo et al., 2025) using single-turn subset due to its complexity. As the benchmark only partially releases the evaluation harness, we adapt our own implementation of the evaluation stack. We report the tool success rate and the benchmark’s average rubric score (ARS).

A small but striking transfer signal. Despite the difficulty of the benchmark, the mid-trained checkpoint shows a consistent agentic advantage over the baseline. It invokes tools more often, executes tools with higher success rate, and achieves marginally better single-turn performance across most grouped domains. Concretely, adding MidTool-Mix improves the tool success rate from 0.5863 to 0.7231, and it also improves the overall rubric score from 0.0567 to 0.0661, with gains in most domains. We therefore view this result as an explosive early signal that general tool-use mid-training may transfer beyond the text-only setting, even without vision-centric tool use pretraining and post-training.

Table 13 | Exploratory results on the single-turn subset of VisualToolBench. We fine-tune the base model on a FineVision subset and then on the same text-only tool-use SFT dataset as in our main setting. We report tool success rate and average rubric score; higher is better. Llama4-Maverick score is reported from the original benchmark paper for reference.

Model	Tool Succ.	Overall	STEM	Med	Fin	Sppt	Gen
Llama4-Maverick*	-	0.1545	0.1875	0.1581	0.1182	0.1562	0.1524
Gemma3-4B-pt + SFT	0.5863	0.0567	0.0655	0.0600	0.0573	0.0564	0.0448
Gemma3-4B-pt + MidTool-Mix+ SFT	0.7231	0.0661	0.0733	0.0765	0.0640	0.0563	0.0606

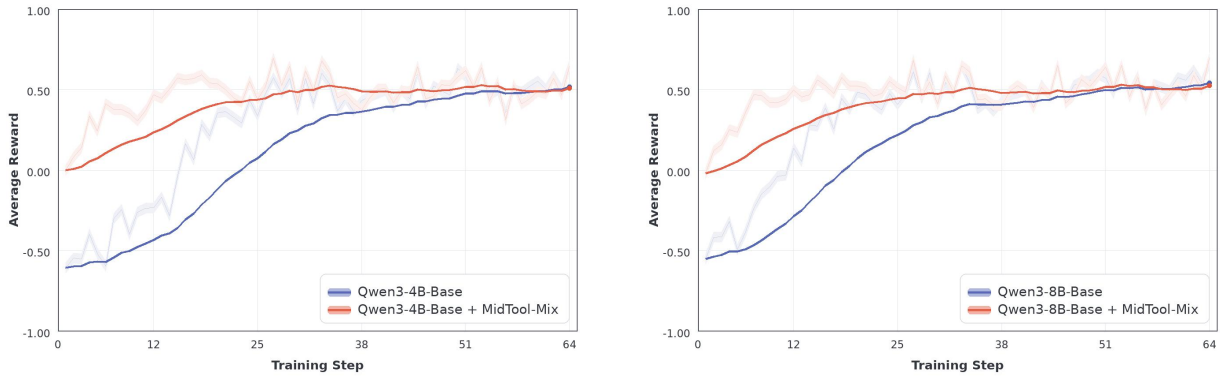


Figure 5 | Average RL reward during training for 4B (left) and 8B (right) models. In both scales, MidTool-Mix starts from a stronger point and improves more quickly in the early stage of RL, while the non-mid-trained baseline gradually catches up later within the same training environment.

Lessons for building stronger agentic intelligence. First, in a substantial portion of VisualTool-Bench, successful tool use requires environment-oriented code fullfill for visual tool invocation (e.g., image processing), and failures often arise at this stage. This code-heavy form of tool use is meaningfully different from the more schema-grounded function-calling and API-style tool use emphasized by our main setting, but is central to CLI-style agency and SWE-oriented agency. Second, even when the model successfully calls a tool, it does not always incorporate the returned evidence into the final answer. This pattern resembles what we observe for web-search tasks in MCP-Universe: the model can trigger the tool, but does not always fully ground the final response in the tool outputs. Taken together, these observations suggest a natural future direction: augmenting mid-training with more code-heavy agentic tool-use data, especially for execution-centric environments, and developing training signals that more explicitly teach the model to ground final answers in tool results, including redundant or partially overlapping evidence returned by multiple tool interactions.

C.4. Illustrative Examples

We show two representative Examples (truncated for space). Figure 6 is a web-derived triplet containing source text, a QA pair, and a tool-augmented trajectory. Figure 7 is a native agentic trajectory stored as a single flattened conversation with interleaved tool calls and responses.

D. Limitations and Future Work

MidTool is a first step toward treating general tool use as a mid-training problem, and the questions it leaves open are, in our view, the most interesting part of this direction.

Co-designing mid-training and post-training. Our experiments deliberately hold the downstream recipe fixed so that the effect of the mid-training corpus can be isolated, and our compute budget did not allow us to also sweep study the post-training axis. The relationship between the two stages is therefore largely unexplored, and we believe it is the most promising next step: how the composition of the SFT corpus interacts with the mid-training mixture, whether stronger or larger post-training supervision substitutes for or compounds with a tool-use prior, and how the choice of RL environments and reward design shifts what the prior is worth. Each of these deserves its own ablation, and we regard them as core questions for mid-training research rather than as details of the present study.

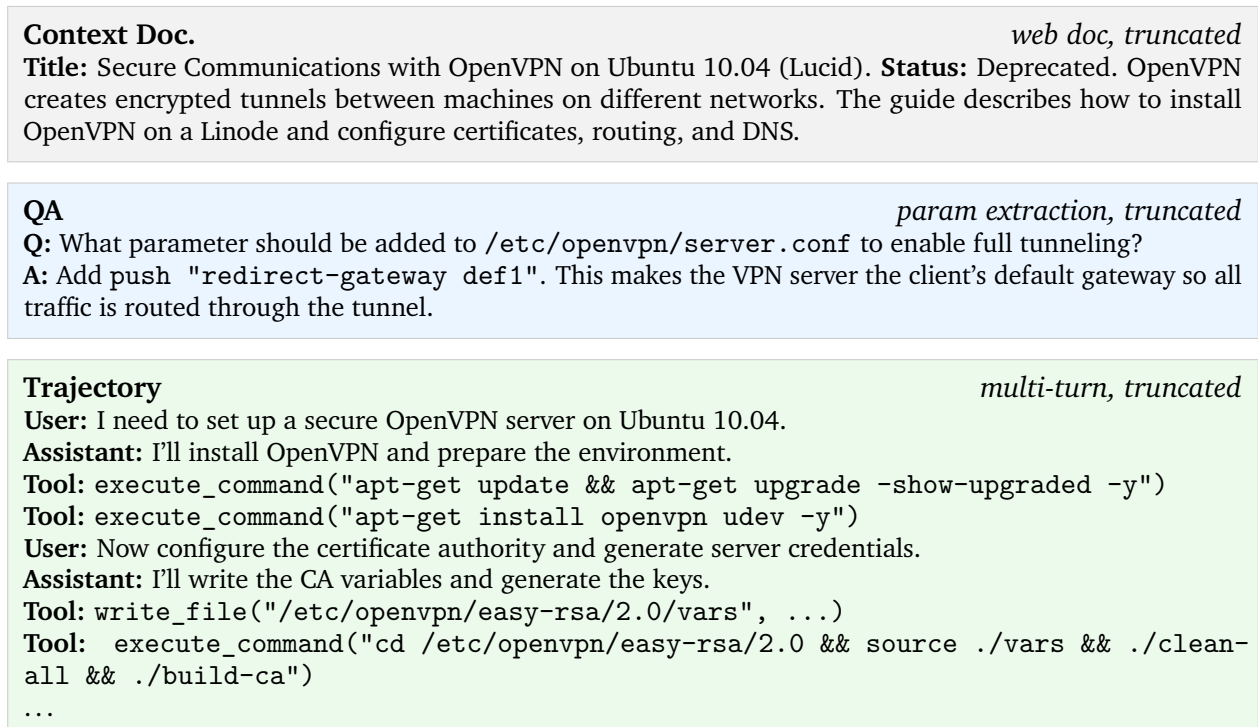


Figure 6 | Illustrative training datum from the web slice. A single example contains (top) a context documentation, (middle) a QA pair derived from that source, and (bottom) a multi-turn trajectory. For readability, long spans are truncated and tool calls are lightly normalized.

Mapping the mixture design space at matched budget. The ablation in Table 6 varies the mixture under a fixed recipe, which already separates raw source signal from synthesized supervision. A fuller account would hold the token budget constant across variants and add controls such as synthesized QA pairs without their originating context, an equal-compute mixture of generic technical documentation, and a native-trajectory-only mixture scaled to the full budget. The last of these is a substantial undertaking on its own, since it requires executable tool environments, interaction generation, and validation at the scale of tens of billions of tokens, and we see it as an interesting systems-and-data challenge in its own right.

Reducing dependence on strong teachers. Both synthesis branches of MidTool currently rely on teacher models. As open-weight agentic models continue to improve, the same pipeline can be instantiated with progressively smaller or self-generated teachers, and the point at which a model can usefully synthesize its own tool-use mid-training data becomes an informative measure of agentic maturity. We expect this question to be most meaningful at larger student scales, where self-generation is diverse and reliable enough to sustain the pipeline's validation requirements.

Beyond general tool use. Finally, our results locate a boundary rather than a ceiling: a general tool-use prior transfers broadly across unfamiliar schemas and MCP domains, while search-heavy and other exploratory behaviors appear to require their own supervision. Extending the mid-training view to deep search, software engineering, and vertical agentic workflows, and understanding which priors are shared across them, is the broader agenda we hope this work helps open.

Native agentic trajectory

truncated

System: You are a helpful assistant with access to the following tools. **Tools:** `tappi.open`, `tappi.elements`, `tappi.click`, `tappi.type`, `tappi.text`, `tappi.screenshot`, ...

User: Reproduce a checkout bug on <https://shop.mountaintrailssupply.com/> where the “Place order” button spins forever.

Assistant: I will open the storefront, follow the customer’s steps, and capture any failure text or screenshot.

Tool: `tappi.open(storefront)`

User: A direct product URL is available; use it, add one item to cart, and confirm the cart text. **Tool:** `tappi.open(product_url)`

Tool: `tappi.elements()`

Tool: `tappi.click(index=2)`

Tool: `tappi.text()`

Assistant: The item is in cart; next I will proceed to checkout, fill dummy data, click “Place order”, then inspect the stuck state. **Tool:** `tappi.click(index=3)`

Tool: `tappi.type(index=0, text=email)`

Tool: `tappi.type(index=5, text=card)`

Tool: `tappi.click(index=8)`

Tool: `tappi.text()`

Tool: `tappi.screenshot(path=artifacts/checkout-spinner.png)`

Tool response: “Checkout ... Processing your order... Please wait while we complete your order ... Place order”

Tool response: screenshot saved to `artifacts/checkout-spinner.png`

Assistant: The page remained on <https://shop.mountaintrailssupply.com/checkout> with the visible message “Processing your order...” and “Please wait while we complete your order ...”. This matches the customer’s report that the checkout button spins indefinitely.

Figure 7 | Illustrative native agentic trajectory example. Long spans are truncated and lightly normalized for readability.